

CSE 331: Computer Security Fundamentals Spring 2026

Reverse Engineering

Amir Rahmati
Xinrui Yu

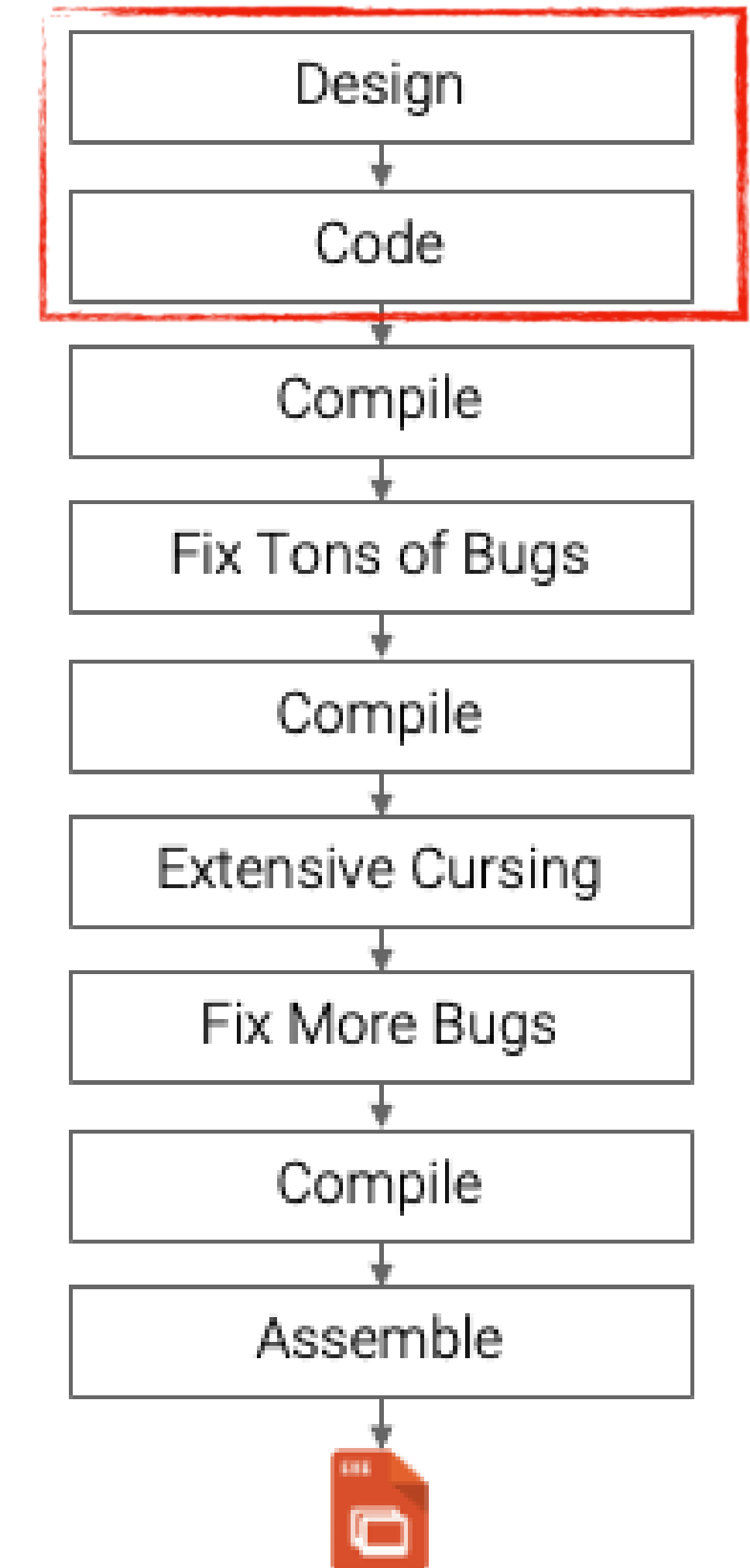


Stony Brook Institute
at Anhui University

安徽大学纽约石溪学院

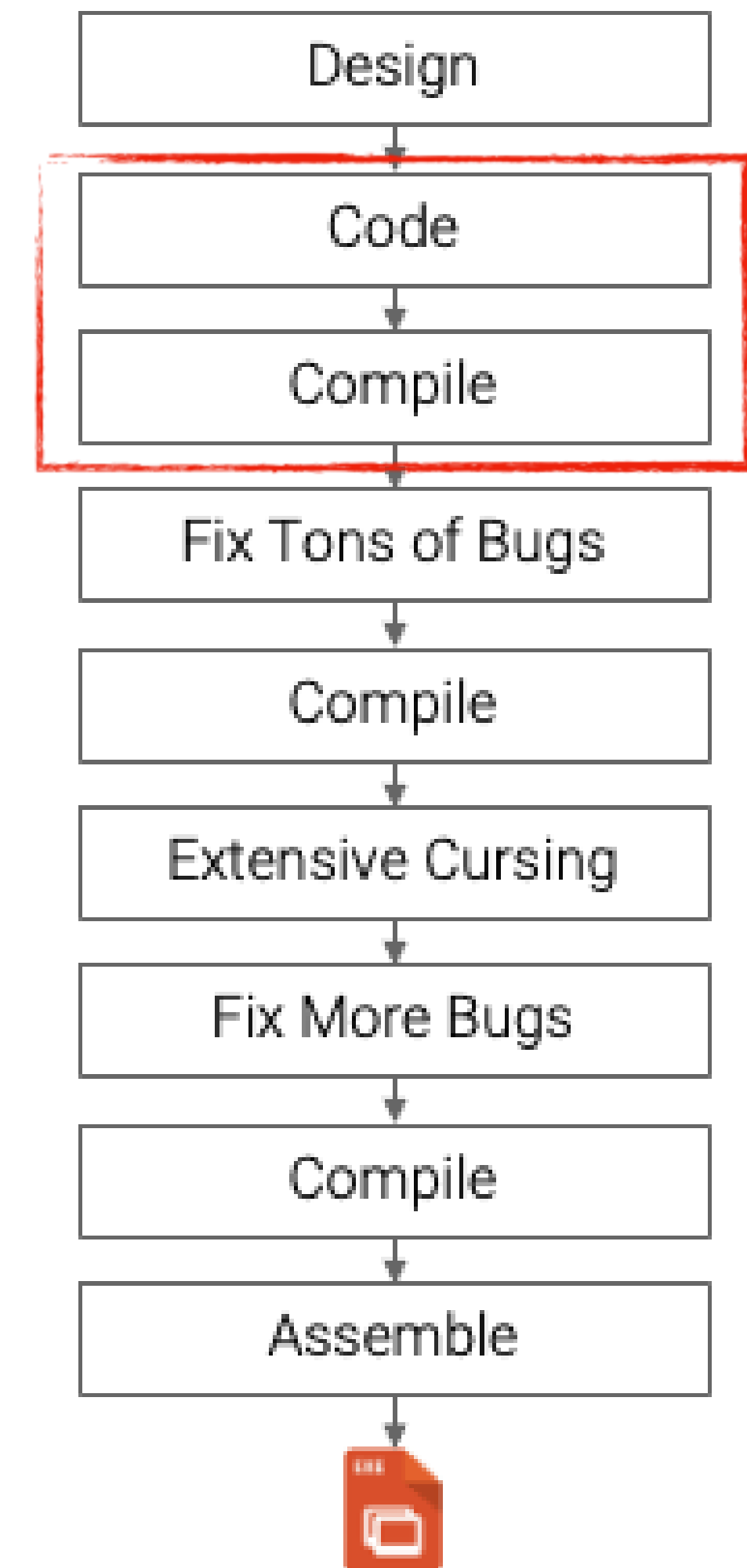
Software Engineering Process

- Usual engineering process:
 - Figure out what you want to code
 - Code it
 - Compile it
 - Run it
- At every step, information is lost!
- Reverse Engineering is the process of getting this info back



Software Engineering Process

- What is lost during compilation?
 - Comments
 - Variable names.
 - Function names.
 - Structure (classes, structs, etc) data.
 - Sometimes, entire algorithms (optimization)!



Simple Example

The image shows the Compiler Explorer interface. On the left, the C++ source code is displayed in a file named 'C++ source #1'. It defines a function `bar` that takes three integers and returns the first, and a `main` function that calls `bar` with arguments 1, 2, and 3, increments the result by 5, and returns it.

```
1 int bar(int arg1, int arg2, int arg3) {  
2     return arg1;  
3 }  
4 int main() {  
5     int local_var = bar(1, 2, 3);  
6     local_var += 5;  
7     return local_var;  
8 }
```

On the right, the assembly output for x86-64 gcc 11.2 is shown. It details the stack frame setup for `bar` and `main`, the function calls, and the final return values.

```
1 bar(int, int, int):  
2     push    rbp  
3     mov     rbp, rsp  
4     mov     DWORD PTR [rbp-4], edi  
5     mov     DWORD PTR [rbp-8], esi  
6     mov     DWORD PTR [rbp-12], edx  
7     mov     eax, DWORD PTR [rbp-4]  
8     pop     rbp  
9     ret  
10 main:  
11     push    rbp  
12     mov     rbp, rsp  
13     sub     rsp, 16  
14     mov     edx, 3  
15     mov     esi, 2  
16     mov     edi, 1  
17     call    bar(int, int, int)  
18     mov     DWORD PTR [rbp-4], eax  
19     add     DWORD PTR [rbp-4], 5  
20     mov     eax, DWORD PTR [rbp-4]  
21     leave  
22     ret
```

The bottom status bar indicates the compilation was successful with x86-64 gcc 11.2, taking 427ms and producing 4410B of output. It also shows that approximately 270 lines were filtered.

Compiler License

Preprocessor

Mac: clang -E hello.c > hello.i

Linux: cpp hello.c > hello.i

- Removal of Comments
- Expansion of Macros
- Expansion of the included files.
- Conditional compilation

```
512 # 32 "/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk/usr/include/secure/_stdio.h" 2 3 4
513 # 42 "/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk/usr/include/secure/_stdio.h" 3 4
514 extern int __sprintf_chk (char * restrict, int, size_t,
515     const char * restrict, ...);
516 # 52 "/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk/usr/include/secure/_stdio.h" 3 4
517 extern int __snprintf_chk (char * restrict, size_t, int, size_t,
518     const char * restrict, ...);
519
520
521
522
523
524
525
526 extern int __vsnprintf_chk (char * restrict, int, size_t,
527     const char * restrict, va_list);
528
529
530
531
532
533
534
535 extern int __vsnprintf_chk (char * restrict, size_t, int, size_t,
536     const char * restrict, va_list);
537 # 408 "/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk/usr/include/stdio.h" 2 3 4
538 # 2 "hello.c" 2
539
540 void say_hello(char *name){
541     printf("hello %s!\n", name);
542 }
543
544 int main(){
545
546     char my_name[1024];
547     scanf("%1000s", my_name);
548     say_hello(my_name);
549 }
550
```


Assembler

Mac: clang -S -mllvm --x86-asm-syntax=intel hello.i

Linux: gcc -S -masm=intel hello.i

- No variable name
- No type information
- Padding
- Even worse: “Strip” removes metadata, function name, ...

myname →

function name still there! →

```
1  .section __TEXT,__text,regular,pure_instructions
2  .build_version macos, 10, 15 sdk_version 10, 15, 6
3  .intel_syntax noprefix
4  .globl _say_hello          ## -- Begin function say_hello
5  .p2align 4, 0x90
6  _say_hello:                ## @say_hello
7  .cfi_startproc
8  ## %bb.0:
9  push rbp
10 .cfi_def_cfa_offset 16
11 .cfi_offset rbp, -16
12 mov rbp, rsp
13 .cfi_def_cfa_register rbp
14 sub rsp, 16
15 mov qword ptr [rbp - 8], rdi
16 mov rsi, qword ptr [rbp - 8]
17 lea rdi, [rip + L_.str]
18 mov al, 0
19 call _printf
20 add rsp, 16
21 pop rbp
22 ret
23 .cfi_endproc
24
25 .globl _main                ## -- Begin function main
26 .p2align 4, 0x90
27 _main:                      ## @main
28 .cfi_startproc
29 ## %bb.0:
30 push rbp
31 .cfi_def_cfa_offset 16
32 .cfi_offset rbp, -16
33 mov rbp, rsp
34 .cfi_def_cfa_register rbp
35 sub rsp, 1056
36 lea rsi, [rbp - 1040]
37 mov rax, qword ptr [rip + __stack_chk_guard@GOTPCREL]
38 mov rax, qword ptr [rax]
39 mov qword ptr [rbp - 8], rax
40 lea rdi, [rip + L_.str.1]
41 mov al, 0
42 call _scanf
43 lea rdi, [rbp - 1040]
44 mov dword ptr [rbp - 1044], eax ## 4-byte Spill
45 call _say_hello
46 mov rcx, qword ptr [rip + __stack_chk_guard@GOTPCREL]
47 mov rcx, qword ptr [rcx]
48 mov rdx, qword ptr [rbp - 8]
49 cmp rcx, rdx
50 jne LBB1_2
51 ## %bb.1:
52 xor eax, eax
53 add rsp, 1056
54 pop rbp
55 ret
56 LBB1_2:
57 call __stack_chk_fail
58 ud2
59 .cfi_endproc
60
61 .section __TEXT,__cstring,cstring_literals
62 L_.str:                      ## @.str
63 .asciz "hello %s!\n"
64
65 L_.str.1:                    ## @.str.1
66 .asciz "%1000s"
67
68 .subsections_via_symbols
```

Things get even worse: Strip

objdump -d a.out

- Typically you don't ship software with all that info
 - Increase size
 - Increase reversibility

```
Disassembly of section __TEXT,__text:

00000000100003ec0: _say_hello:
100003ec0: 55                pushq   %rbp
100003ec1: 48 89 e5          movq    %rsp, %rbp
100003ec4: 48 83 ec 10       subq    $16, %rsp
100003ec8: 48 89 7d f8       movq    %rdi, -8(%rbp)
100003ecc: 48 8b 75 f8       movq    -8(%rbp), %rsi
100003ed0: 48 8d 3d c3 00 00 00 leaq    195(%rip), %rdi
100003ed7: b0 00           movb    $0, %al
100003ed9: e8 82 00 00 00   callq   130 <dyld_stub_binder+0x100003f60>
100003ede: 48 83 c4 10       addq    $16, %rsp
100003ee2: 5d              popq    %rbp
100003ee3: c3              retq
100003ee4: 66 2e 0f 1f 84 00 00 00 00 00 nopw    %cs:(%rax,%rax)
100003eee: 66 90           nop

00000000100003ef0: _main:
100003ef0: 55                pushq   %rbp
100003ef1: 48 89 e5          movq    %rsp, %rbp
100003ef4: 48 81 ec 20 04 00 00 subq    $1056, %rsp
100003efb: 48 8d b5 f0 fb ff ff leaq    -1040(%rbp), %rsi
100003f02: 48 8b 05 f7 00 00 00 movq    247(%rip), %rax
100003f09: 48 8b 00          movq    (%rax), %rax
100003f0c: 48 89 45 f8       movq    %rax, -8(%rbp)
100003f10: 48 8d 3d 8e 00 00 00 leaq    142(%rip), %rdi
100003f17: b0 00           movb    $0, %al
100003f19: e8 48 00 00 00   callq   72 <dyld_stub_binder+0x100003f66>
100003f1e: 48 8d bd f0 fb ff ff leaq    -1040(%rbp), %rdi
100003f25: 89 85 ec fb ff ff movl    %eax, -1044(%rbp)
100003f2b: e8 90 ff ff ff   callq   -112 <_say_hello>
100003f30: 48 8b 0d c9 00 00 00 movq    201(%rip), %rcx
100003f37: 48 8b 09          movq    (%rcx), %rcx
100003f3a: 48 8b 55 f8       movq    -8(%rbp), %rdx
100003f3e: 48 39 d1          cmpq    %rdx, %rcx
100003f41: 0f 85 0b 00 00 00 jne     11 <_main+0x62>
100003f47: 31 c0            xorl    %eax, %eax
100003f49: 48 81 c4 20 04 00 00 addq    $1056, %rsp
100003f50: 5d              popq    %rbp
100003f51: c3              retq
100003f52: e8 03 00 00 00   callq   3 <dyld_stub_binder+0x100003f5a>
100003f57: 0f 0b           ud2

Disassembly of section __TEXT,__stubs:

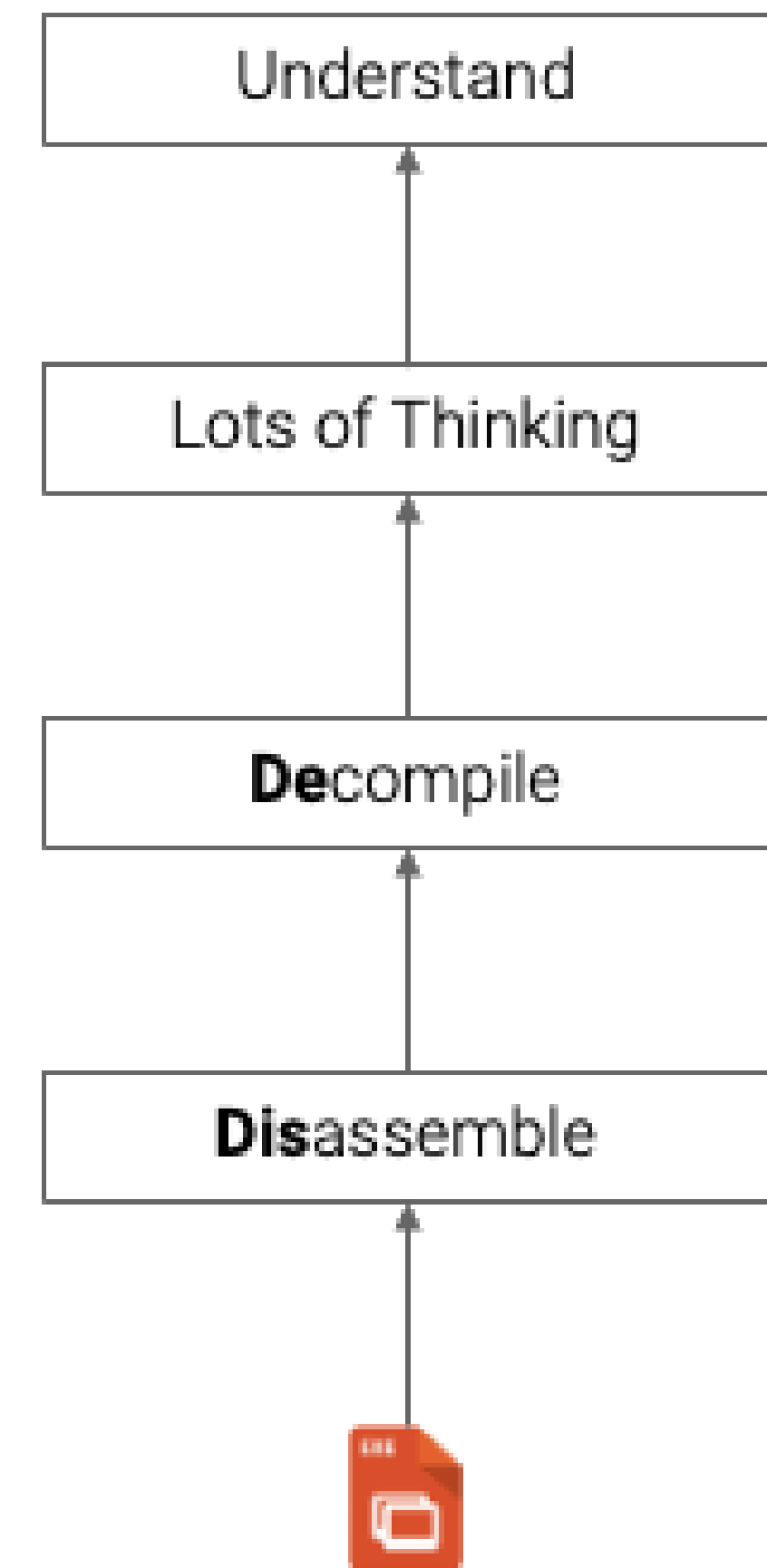
00000000100003f5a: __stubs:
100003f5a: ff 25 a0 40 00 00 jmpq    *16544(%rip)
100003f60: ff 25 a2 40 00 00 jmpq    *16546(%rip)
100003f66: ff 25 a4 40 00 00 jmpq    *16548(%rip)

Disassembly of section __TEXT,__stub_helper:

00000000100003f6c: __stub_helper:
100003f6c: 4c 8d 1d a5 40 00 00 leaq    16549(%rip), %r11
100003f73: 41 53           pushq   %r11
100003f75: ff 25 8d 00 00 00 jmpq    *141(%rip)
100003f7b: 90              nop
100003f7c: 68 00 00 00 00 00 pushq   $0
100003f81: e9 e6 ff ff ff   jmp     -26 <__stub_helper>
100003f86: 68 18 00 00 00 00 pushq   $24
100003f8b: e9 dc ff ff ff   jmp     -36 <__stub_helper>
100003f90: 68 26 00 00 00 00 pushq   $38
100003f95: e9 d2 ff ff ff   jmp     -46 <__stub_helper>
```

Reverse Engineering Process

- Every step in the reverse-engineering process is imperfect and relies on some amount of human help.
- Typically, a reverser use several reverse engineering tools to build up a mental model of the target software.
- How do we reverse the design from the binary?



What is a program?

- A **program**...
 - consists of **modules**...
 - that are made up of **functions**...
 - that contain **blocks**...
 - of **instructions**...
 - that operate on **variables** and **data structures**.

What is a module?

- Developers (frequently) rely on libraries to build software. These libraries have well-documented functionality!
- No need to reverse engineer library codes!
- Read The Fine Manual of the libraries, and focus your reversing effort on your actual target!

```
yans@pwn ~ $ /bin/cat /proc/self/maps
561de3fad000-561de3faf000 r--p 00000000 fd:01 7864471 /usr/bin/cat
561de3faf000-561de3fb4000 r-xp 00002000 fd:01 7864471 /usr/bin/cat
561de3fb4000-561de3fb7000 r--p 00007000 fd:01 7864471 /usr/bin/cat
561de3fb7000-561de3fb8000 r--p 00009000 fd:01 7864471 /usr/bin/cat
561de3fb8000-561de3fb9000 rw-p 0000a000 fd:01 7864471 /usr/bin/cat
561de4273000-561de4294000 rw-p 00000000 00:00 0 [heap]
7f10139d4000-7f10139f6000 rw-p 00000000 00:00 0
7f10139f6000-7f10147d4000 r--p 00000000 fd:01 7870618 /usr/lib/locale/locale-archive
7f10147d4000-7f10147f9000 r--p 00000000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10147f9000-7f1014971000 r-xp 00025000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f1014971000-7f10149bb000 r--p 0019d000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bb000-7f10149bc000 ---p 001e7000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bc000-7f10149bf000 r--p 001e7000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bf000-7f10149c2000 rw-p 001ea000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149c2000-7f10149c8000 rw-p 00000000 00:00 0
7f10149e6000-7f10149e7000 r--p 00000000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f10149e7000-7f1014a0a000 r-xp 00001000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a0a000-7f1014a12000 r--p 00024000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a13000-7f1014a14000 r--p 0002c000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a14000-7f1014a15000 rw-p 0002d000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a15000-7f1014a16000 rw-p 00000000 00:00 0
7ffd5d20a000-7ffd5d22c000 rw-p 00000000 00:00 0 [stack]
7ffd5d3a0000-7ffd5d3a3000 r--p 00000000 00:00 0 [vvar]
7ffd5d3a3000-7ffd5d3a4000 r-xp 00000000 00:00 0 [vdso]
fffffffff6000000-fffffffff6010000 --xp 00000000 00:00 0 [vsyscall]
```

What is a function?

- Functions represent fairly well-encapsulated functionality.
- Most functions have a well-defined goal such as:
 - set some data
 - calculate/retrieve/validate data
 - dispatch other functions
 - perform some action on the outside world (via system calls)
- Initially, functions can be reverse-engineered in isolation. Later, you can build up an understanding of how they fit together.

Control Flow Graph (CFG)

- Functions are represented as a **graph**.
- Each **block** is a set of instructions that will execute one after the other.
- Blocks are joined by **edges**, representing conditional and unconditional jumps.
- By understanding what the blocks do and what conditions trigger what edges, you can understand the function's logic!

```
_start:  
0040104c  mov     rdi, qword [rsp {__return_addr}]  
00401050  call    main  
00401055  mov     rax, 0x3c  
0040105c  mov     rdi, 0x0  
00401063  syscall  
{ Does not return }
```

```
main:  
00401000  push    rbp {__saved_rbp}  
00401001  mov     rbp, rsp {__saved_rbp}  
00401004  sub     rsp, 0x100
```

```
0040100b  mov     rax, 0x0  
00401012  mov     rdi, 0x0  
00401019  mov     rsi, rsp {var_108}  
0040101c  mov     rdx, 0x100  
00401023  syscall  
00401025  cmp     rax, 0x0  
00401029  jle     .ret
```

```
00401047  mov     rsp, rbp  
0040104a  pop     rbp {__saved_rbp}  
0040104b  retn    {__return_addr}
```

```
0040102b  mov     rax, 0x1  
00401032  mov     rdi, 0x1  
00401039  mov     rsi, rsp {var_108}  
0040103c  mov     rdx, 0x100  
00401043  syscall  
00401045  jmp     .read
```



Control Flow Graph (CFG)

- Functions often begin with a prologue and end with an epilogue.
- The stack frame

```
_start:  
0040104c mov     rdi, qword [rsp {__return_addr}]  
00401050 call    main  
00401055 mov     rax, 0x3c  
0040105c mov     rdi, 0x0  
00401063 syscall  
{ Does not return }
```

```
main:  
00401000 push    rbp {__saved_rbp}  
00401001 mov     rbp, rsp {__saved_rbp}  
00401004 sub     rsp, 0x100
```

Set up

```
0040100b mov     rax, 0x0  
00401012 mov     rdi, 0x0  
00401019 mov     rsi, rsp {var_108}  
0040101c mov     rdx, 0x100  
00401023 syscall  
00401025 cmp     rax, 0x0  
00401029 jle     .ret
```

tear down

```
00401047 mov     rsp, rbp  
0040104a pop     rbp {__saved_rbp}  
0040104b retn    {__return_addr}
```

```
0040102b mov     rax, 0x1  
00401032 mov     rdi, 0x1  
00401039 mov     rsi, rsp {var_108}  
0040103c mov     rdx, 0x100  
00401043 syscall  
00401045 jmp     .read
```

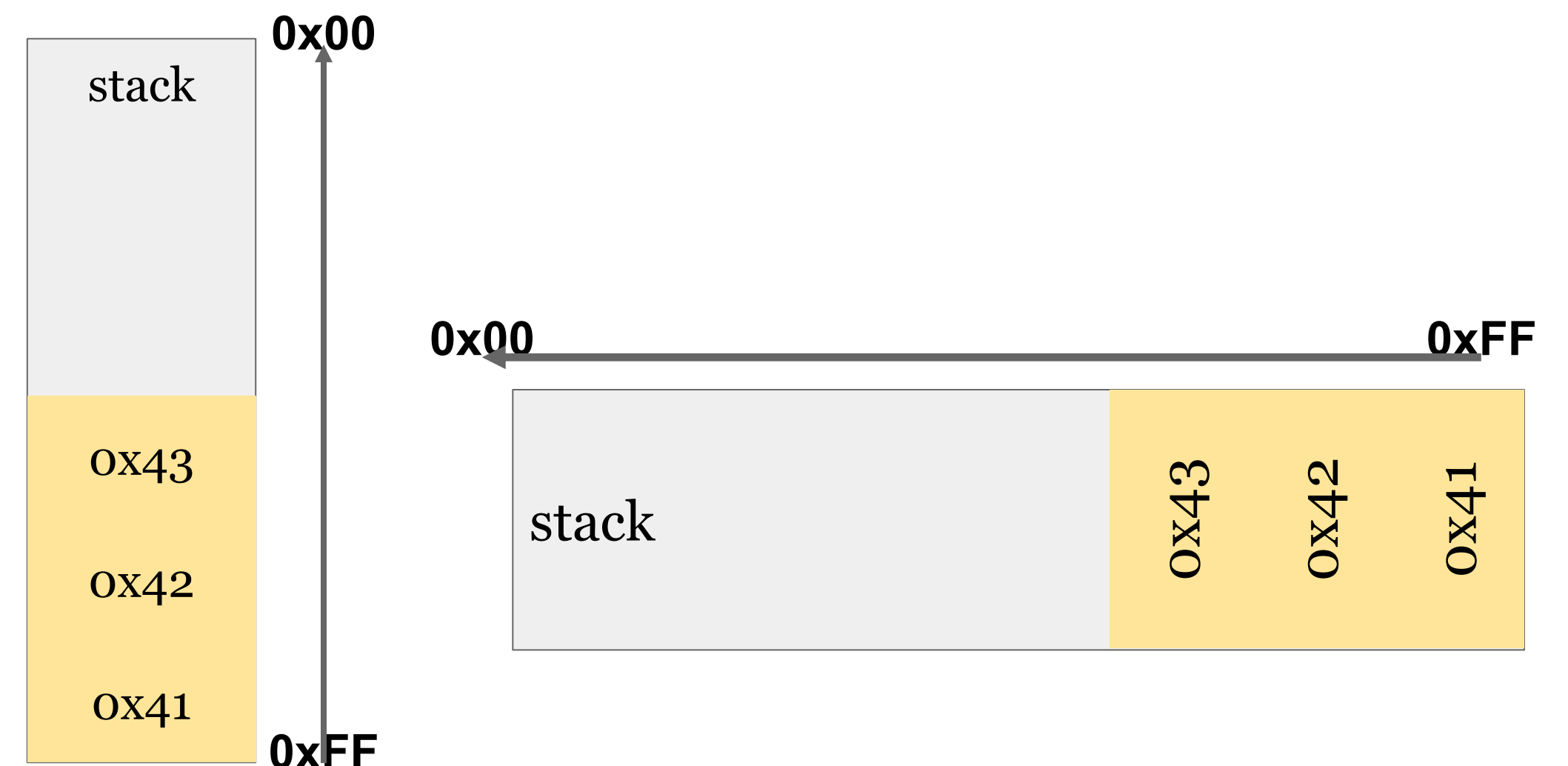

Where are variables stored?

- Global variables
 - **.data:** used for pre-initialized global writable data (such as global arrays with initial values)
 - **.rodata:** used for global read-only data (such as string constants)
 - **.bss:** used for uninitialized global writable data (such as global arrays without initial values)
- Local variables
 - Stack!

The Stack

- region of memory used to store local variables and call contexts.
- the stack grows backwards.
- When you push to the stack, `rsp` is decreased by 8.
- When you pop from the stack, `rsp` is increased by 8.
- push 0x41; push 0x42; push 0x43

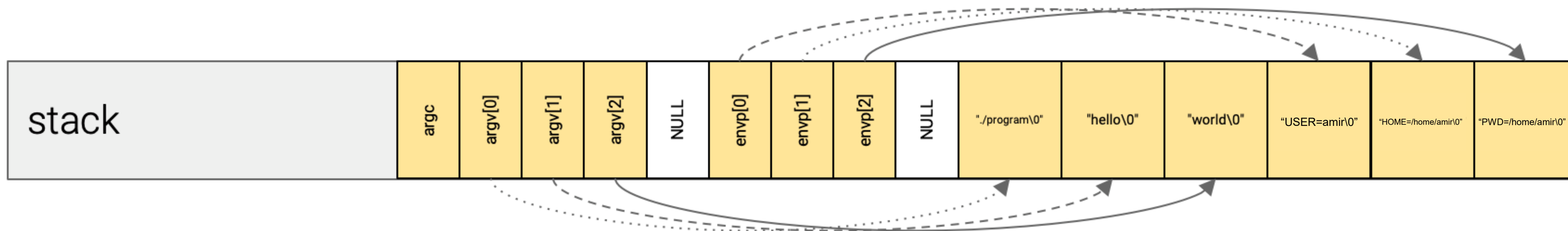
```
561de3fad000-561de3faf000 r--p 00000000 fd:01 7864471 /usr/bin/cat
561de3faf000-561de3fb4000 r-xp 00002000 fd:01 7864471 /usr/bin/cat
561de3fb4000-561de3fb7000 r--p 00007000 fd:01 7864471 /usr/bin/cat
561de3fb7000-561de3fb8000 r--p 00009000 fd:01 7864471 /usr/bin/cat
561de3fb8000-561de3fb9000 rw-p 0000a000 fd:01 7864471 /usr/bin/cat
561de4273000-561de4294000 rw-p 00000000 00:00 0 [heap]
7f10139d4000-7f10139f6000 rw-p 00000000 00:00 0
7f10139f6000-7f10147d4000 r--p 00000000 fd:01 7870618 /usr/lib/locale/locale-archive
7f10147d4000-7f10147f9000 r--p 00000000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10147f9000-7f1014971000 r-xp 00025000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f1014971000-7f10149bb000 r--p 0019d000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bb000-7f10149bc000 ---p 001e7000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bc000-7f10149bf000 r--p 001e7000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bf000-7f10149c2000 rw-p 001ea000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149c2000-7f10149c8000 rw-p 00000000 00:00 0
7f10149e6000-7f10149e7000 r--p 00000000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f10149e7000-7f1014a0a000 r-xp 00001000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a0a000-7f1014a12000 r--p 00024000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a13000-7f1014a14000 r--p 0002c000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a14000-7f1014a15000 rw-p 0002d000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a15000-7f1014a16000 rw-p 00000000 00:00 0
7fffd5d20a000-7fffd5d22c000 rw-p 00000000 00:00 0 [stack]
7fffd5d3a0000-7fffd5d3a3000 r--p 00000000 00:00 0 [vvar]
7fffd5d3a3000-7fffd5d3a4000 r-xp 00000000 00:00 0 [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```



Environments and Arguments

The stack starts out storing (among some other things) the environment variables and the program arguments. For example:

```
$ env  
USER=amir  
HOME=/home/amir  
PWD=/home/amir  
$ ./program hello world  
hello world
```



Calling a Function

- When a function is called, the address that the called function should return to is implicitly pushed onto the stack.
- This return address is implicitly popped when the function returns.

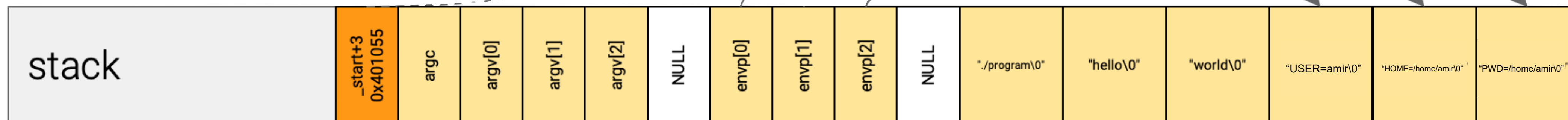
```
_start:  
0040104c mov     rdi, qword [rsp {__return_addr}]  
00401050 call    main  
00401055 mov     rax, 0x3c  
0040105c mov     rdi, 0x0  
00401063 syscall  
{ Does not return }
```

```
main:  
00401000 push    rbp {__saved_rbp}  
00401001 mov     rbp, rsp {__saved_rbp}  
00401004 sub     rsp, 0x100
```

```
0040100b mov     rax, 0x0  
00401012 mov     rdi, 0x0  
00401019 mov     rsi, rsp {var_108}  
0040101c mov     rdx, 0x100  
00401023 syscall  
00401025 cmp     rax, 0x0  
00401029 jle     .ret
```

```
00401047 mov     rsp, rbp  
0040104a pop     rbp {__saved_rbp}  
0040104b retn    {__return_addr}
```

```
0040102b mov     rax, 0x1  
00401032 mov     rdi, 0x1  
00401039 mov     rsi, rsp {var_108}  
0040103c mov     rdx, 0x100  
00401043 syscall  
00401045 jmp     .read
```



Function Frame Setup

- Every function sets up its stack frame. It has:
 - **Stack pointer (rsp)**: points to the leftmost side of the stack frame.
 - **Base pointer (rbp)**: points to the rightmost side of the stack frame.
- **Prologue**:
 - save off the caller's base pointer
 - set the current stack pointer as the base pointer
 - “allocate” space on stack (subtract from the stack pointer)

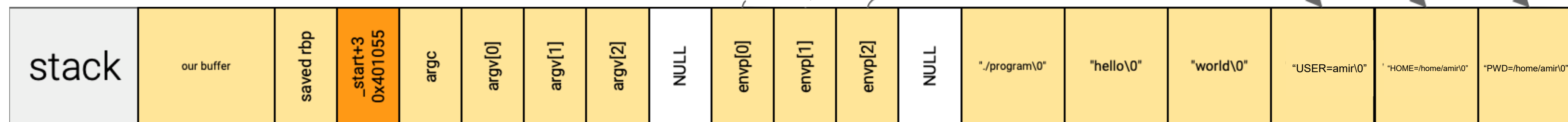
```
_start:
0040104c  mov     rdi, qword [rsp {__return_addr}]
00401050  call    main
00401055  mov     rax, 0x3c
0040105c  mov     rdi, 0x0
00401063  syscall
{ Does not return }
```

```
00401000  push    rbp {__saved_rbp}
00401001  mov     rbp, rsp {__saved_rbp}
00401004  sub     rsp, 0x100
```

```
0040100b  mov     rax, 0x0
00401012  mov     rdi, 0x0
00401019  mov     rsi, rsp {var_108}
0040101c  mov     rdx, 0x100
00401023  syscall
00401025  cmp     rax, 0x0
00401029  jle     .ret
```

```
00401047  mov     rsp, rbp
0040104a  pop     rbp {__saved_rbp}
0040104b  retn    {__return_addr}
```

```
0040102b  mov     rax, 0x1
00401032  mov     rdi, 0x1
00401039  mov     rsi, rsp {var_108}
0040103c  mov     rdx, 0x100
00401043  syscall
00401045  jmp     .read
```



Function Frame Teardown

```
_start:
0040104c  mov     rdi, qword [rsp {__return_addr}]
00401050  call    main
00401055  mov     rax, 0x3c
0040105c  mov     rdi, 0x0
00401063  syscall
{ Does not return }
```

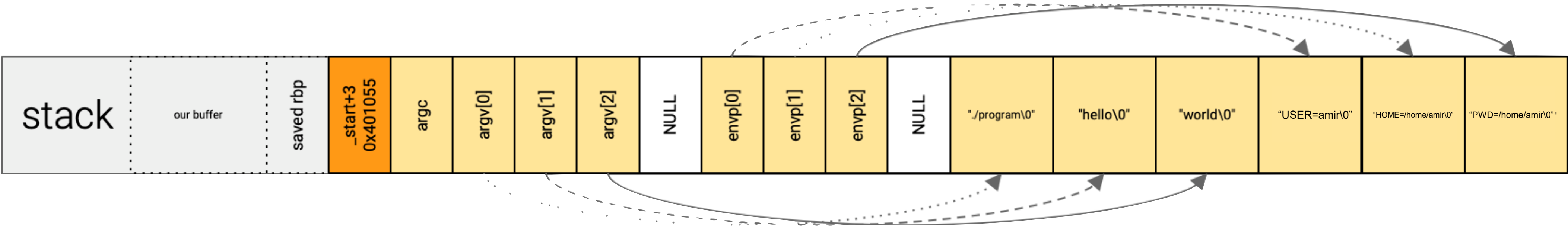
```
main:
00401000  push    rbp {__saved_rbp}
00401001  mov     rbp, rsp {__saved_rbp}
00401004  sub     rsp, 0x100
```

```
0040100b  mov     rax, 0x0
00401012  mov     rdi, 0x0
00401019  mov     rsi, rsp {var_108}
0040101c  mov     rdx, 0x100
00401023  syscall
00401025  cmp     rax, 0x0
00401029  jle     .ret
```

```
00401047  mov     rsp, rbp
0040104a  pop     rbp {__saved_rbp}
0040104b  retn    {__return_addr}

0040102b  mov     rax, 0x1
00401032  mov     rdi, 0x1
00401039  mov     rsi, rsp {var_108}
0040103c  mov     rdx, 0x100
00401043  syscall
00401045  jmp     .read
```

- Every function sets up its stack frame. It has:
 - **Stack pointer (rsp)**: points to the leftmost side of the stack frame.
 - **Base pointer (rbp)**: points to the rightmost side of the stack frame.
- **Epilogue**:
 - "deallocate" the stack (mov rsp, rbp). the data is **NOT** destroyed by default!
 - restore the old base pointer
 - Now, we are ready to return!



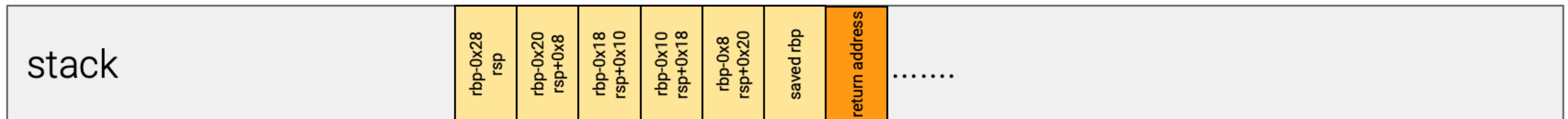
Data storage locations

- Data can be in:
 - **.data:** used for pre-initialized global writable data (such as global arrays with initial values)
 - **.rodata:** used for global read-only data (such as string constants)
 - **.bss:** used for uninitialized global writable data (such as global arrays without initial values)
 - **stack:** used for statically-allocated local variables
 - **heap:** used for dynamically-allocated (malloc()) variables.
 - Let's look at how this data is accessed!

```
561de3fad000-561de3faf000 r--p 00000000 fd:01 7864471 /usr/bin/cat
561de3faf000-561de3fb4000 r-xp 00002000 fd:01 7864471 /usr/bin/cat
561de3fb4000-561de3fb7000 r--p 00007000 fd:01 7864471 /usr/bin/cat
561de3fb7000-561de3fb8000 r--p 00009000 fd:01 7864471 /usr/bin/cat
561de3fb8000-561de3fb9000 rw-p 0000a000 fd:01 7864471 /usr/bin/cat
561de4273000-561de4294000 rw-p 00000000 00:00 0 [heap]
7f10139d4000-7f10139f6000 rw-p 00000000 00:00 0
7f10139f6000-7f10147d4000 r--p 00000000 fd:01 7870618 /usr/lib/locale/locale-archive
7f10147d4000-7f10147f9000 r--p 00000000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10147f9000-7f1014971000 r-xp 00025000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f1014971000-7f10149bb000 r--p 0019d000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bb000-7f10149bc000 ---p 001e7000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bc000-7f10149bf000 r--p 001e7000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149bf000-7f10149c2000 rw-p 001ea000 fd:01 7871432 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f10149c2000-7f10149c8000 rw-p 00000000 00:00 0
7f10149e6000-7f10149e7000 r--p 00000000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f10149e7000-7f1014a0a000 r-xp 00001000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a0a000-7f1014a12000 r--p 00024000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a13000-7f1014a14000 r--p 0002c000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a14000-7f1014a15000 rw-p 0002d000 fd:01 7871221 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f1014a15000-7f1014a16000 rw-p 00000000 00:00 0
7fffd5d20a000-7fffd5d22c000 rw-p 00000000 00:00 0 [stack]
7fffd5d3a0000-7fffd5d3a3000 r--p 00000000 00:00 0 [vvar]
7fffd5d3a3000-7fffd5d3a4000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

Accessing stack data

- **push** (to store data on "top"/left of the stack)
- **pop** (to retrieve data from the "top"/left of the stack)
- **rsp**-relative accesses:
 - load: `mov rdx, [rsp+0x10]`
 - store: `mov [rsp+0x10], rdx`
- **rbp**-relative accesses:
 - load: `mov rdx, [rbp-0x10]`
 - store: `mov [rbp-0x10], rdx`
- **via reference**
 - `mov rdx, rsp; mov [rdx], 0x41;`
 - `mov rdx, rbp; mov [rdx], 0x41;`



Accessing .data .rodata .bss

- Data in .bss, .rodata, and .data is stored at known offsets from the program code.
- It is accessed via **rip**-relative instructions.
- **Load:** `mov rax, [rip+0x20040]`
- **Store:** `mov [rip+0x20040], rax`
- **Reference:** `lea rax, [rip+0x20040]`

Accessing heap data

- Pointers to heap data are generally stored in memory or on the stack.
- Mind the difference between:

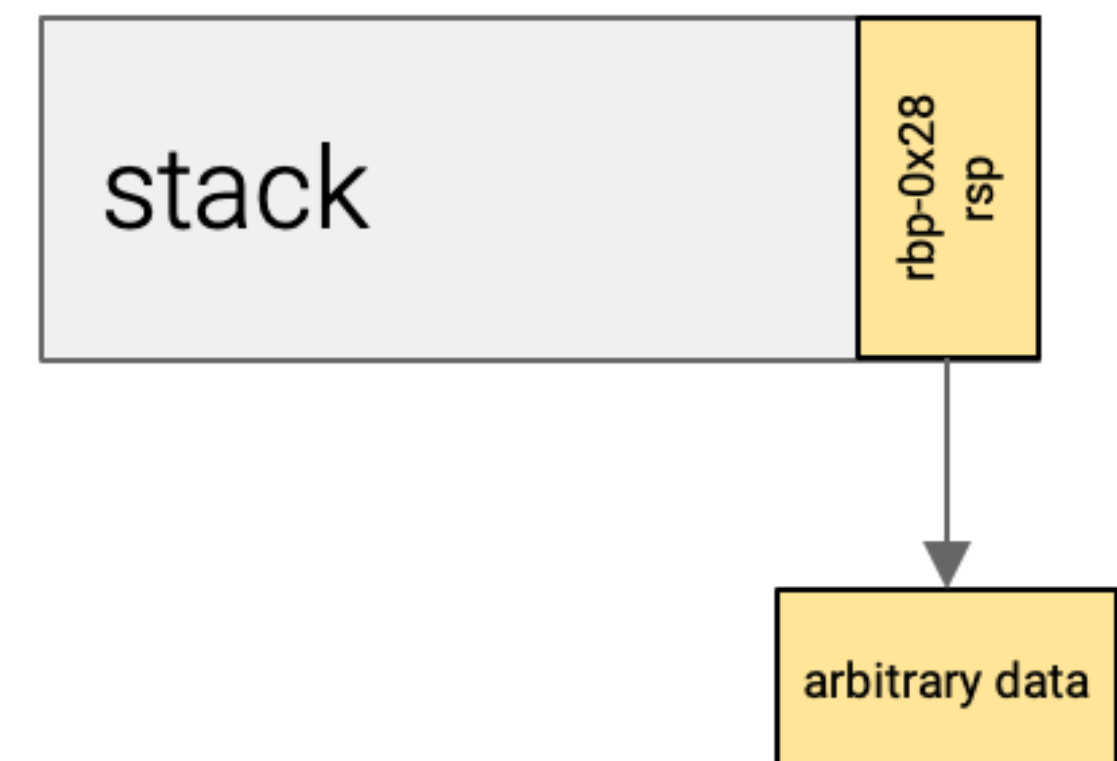
- **Stack data access:**

- `mov rax, rsp`
- `mov rdx, [rax]`



- **Arbitrary data access via stack-stored reference:**

- `mov rax, [rsp]`
- `mov rdx, [rax]`



Data structures

Type information is lost in the compilation process!

You will have to piece type information together based on how data is used by code.

Static Tools

- Static (as opposed to dynamic) reverse engineering: analyzing a program at rest.
- Simple Stuff:
 - **kaitai struct** (<https://ide.kaitai.io/>): file format parser and explorer
 - **nm**: lists symbols used/provided by ELF files
 - **strings**: dumps ASCII (and other format) strings found in a file
 - **objdump**: simple disassembler
 - **checksec** (<https://github.com/slimm609/checksec.sh>): analyzes security features used by an executable

kaitai struct

- (<https://ide.kaitai.io/>): file format parser and explorer

The screenshot displays the Kaitai Struct IDE interface, which is used for parsing and exploring file formats. The interface is divided into several panels:

- Files:** A sidebar on the left showing a file tree with categories like 'formats', 'archive', 'executable', 'filesystem', 'game', 'hardware', 'image', 'media', 'network', 'serialization', and 'samples'. The 'png.ksy' file is selected under the 'image' category.
- .ksy editor:** The main editor showing the Kaitai Struct definition for a PNG file. The definition includes metadata (id: png, file-extension: png, endian: be), a sequence of chunks (magic, ihdr, and a repeatable chunk type), and types for the chunks (IHDR and IDAT).
- parsed as tree:** A panel below the .ksy editor showing the parsed tree structure of the file. It displays the hierarchy of the file, including the magic, ihdrLen, ihdrType, ihdr, and chunks. The ihdr chunk is expanded, showing its fields: width, height, bitDepth, colorType, compressionMethod, filterMethod, and interlaceMethod. The ihdrCrc is also shown.
- hex viewer:** A panel on the right showing the raw bytes of the file in hexadecimal and ASCII. The first few bytes are visible, including the PNG signature and the IHDR chunk.
- info panel:** A panel at the bottom left showing the selected chunk (ihdrCrc) and its value (246, 31, 25, 34).
- converter:** A panel at the bottom right showing a table of values and their conversions. The table has columns for 'Type', 'Value (unsigned)', and '(signed)'. The values are converted from the selected chunk (ihdrCrc) to various data types.

nm

- lists symbols used/provided by ELF files
- Virtual address of the symbol
- A character which depicts the symbol type. Name of the symbol
 - A/a : absolute symbol.
 - B/b : bss symbol.
 - D/d : data symbol.
 - f : Source file name symbol.
 - L/l : thread-local symbol (TLS).
 - T/t : text symbol.
 - U : Undefined symbol.

```
$ nm -A ./*.o | grep func

./hello2.o:0000000000000000 T func_1
./hello3.o:0000000000000000 T func_2
./hello4.o:0000000000000000 T func_3
./main.o:                                U func
./reloc.o:                                U func
./reloc.o:0000000000000000 T func1
./test1.o:0000000000000000 T func
./test.o:                                U func
```

strings

- dumps ASCII (and other format) strings found in a file Virtual address of the symbol
- by default: Longer than 4 chars

```
.rel.plt  
.init  
.text  
.fini  
.rodata  
.eh_frame_hdr  
.eh_frame  
.init_array  
.fini_array  
.dynamic  
.got  
.got.plt  
.data  
.bss  
.comment  
.debug_aranges  
.debug_info  
.debug_abbrev  
.debug_line  
.debug_str
```

objdump

- simple disassembler

```
4004ed    <main>:
4004ed:  55                      push    rbp
4004ee:  48 89 e5                mov     rbp, rsp
4004f1:  c7 45 ec 00 00 00 00    mov     DWORD PTR [rbp-0x14], 0x0
4004f8:  c7 45 f0 01 00 00 00    mov     DWORD PTR [rbp-0x10], 0x1
4004ff:  c7 45 f4 02 00 00 00    mov     DWORD PTR [rbp-0xc], 0x2
400506:  c7 45 f8 03 00 00 00    mov     DWORD PTR [rbp-0x8], 0x3
40050d:  c7 45 fc 04 00 00 00    mov     DWORD PTR [rbp-0x4], 0x4
400514:  c7 45 ec 00 00 00 00    mov     DWORD PTR [rbp-0x14], 0x0
40051b:  eb 13                  jmp     400530 <main+0x43>
40051d:  8b 05 15 0b 20 00       mov     eax, DWORD PTR [rip+0x200b15] # 601038 <globalA>
400523:  83 e8 01                sub     eax, 0x1
400526:  89 05 0c 0b 20 00       mov     DWORD PTR [rip+0x200b0c], eax # 601038 <globalA>
40052c:  83 45 ec 01                add     DWORD PTR [rbp-0x14], 0x1
400530:  8b 05 02 0b 20 00       mov     eax, DWORD PTR [rip+0x200b02] # 601038 <globalA>
400536:  39 45 ec                cmp     DWORD PTR [rbp-0x14], eax
400539:  7c e2                  jl      40051d <main+0x30>
40053b:  5d                      pop     rbp
40053c:  c3                      ret
40053d:  0f 1f 00                nop     DWORD PTR [rax]
```


checksec

- bash script to check the properties of executables (like PIE, RELRO, Canaries, ASLR, Fortify Source)

```
$checksec --file=/bin/ls
RELRO          STACK CANARY      NX            PIE            RPATH          RUNPATH         FILE
Partial RELRO  Canary found      NX enabled    No PIE         No RPATH       No RUNPATH      /bin/ls
```


Static Tools

- Static (as opposed to dynamic) reverse engineering: analyzing a program at rest.
- Simple Stuff:
 - **kaitai struct** (<https://ide.kaitai.io/>): file format parser and explorer
 - **nm**: lists symbols used/provided by ELF files
 - **strings**: dumps ASCII (and other format) strings found in a file
 - **objdump**: simple disassembler
 - **checksec** (<https://github.com/slimm609/checksec.sh>): analyzes security features used by an executable
- Commercial:
 - **IDA Pro**: the "gold standard" of disassemblers (<https://www.hex-rays.com/products/ida/>)
 - **Binary Ninja**: IDA's main commercial competitor (<https://binary.ninja/>)
- Free:
 - **Binary Ninja Cloud**: a version of Binary Ninja that runs in your browser! (<https://cloud.binary.ninja/>)
- Open source:
 - **angr management**: an academic binary analysis framework (<https://github.com/angr/angr-management/releases>)
 - **ghidra**: a reversing tool created by the National Security Agency (<https://ghidra-sre.org/>)
 - **cutter**: a reversing tool created by the radare2 open source project (<https://cutter.re/>)

Dynamic Tools

- Dynamic (as opposed to static) reverse engineering: analyzing a program at runtime.
- **ltrace/strace** traces library/system calls
- Running the program with multiple different inputs might get you farther.
 - ltrace the program with input A
 - ltrace the program again with input B
 - see if you can reverse the algorithm from looking at input and output
 - still does not scale to complex algorithms
- Debugging tools: **gdb**
- **gdb scripting** automates debugging tasks

Timeless Debugging

- Timeless debugging frees you from having to think of breakpoints ahead of time.
 - record execution
 - rewind execution
 - replay execution
- Relevant tools:
 - **gdb** has built-in record-replay functionality (<https://sourceware.org/gdb/current/onlinedocs/gdb/Process-Record-and-Replay.html>)
 - **rr** is a highly-performant record-replay engine (<https://github.com/mozilla/rr>)
 - **qira** is a timeless debugger made for reverse engineering (<https://qira.me/>)

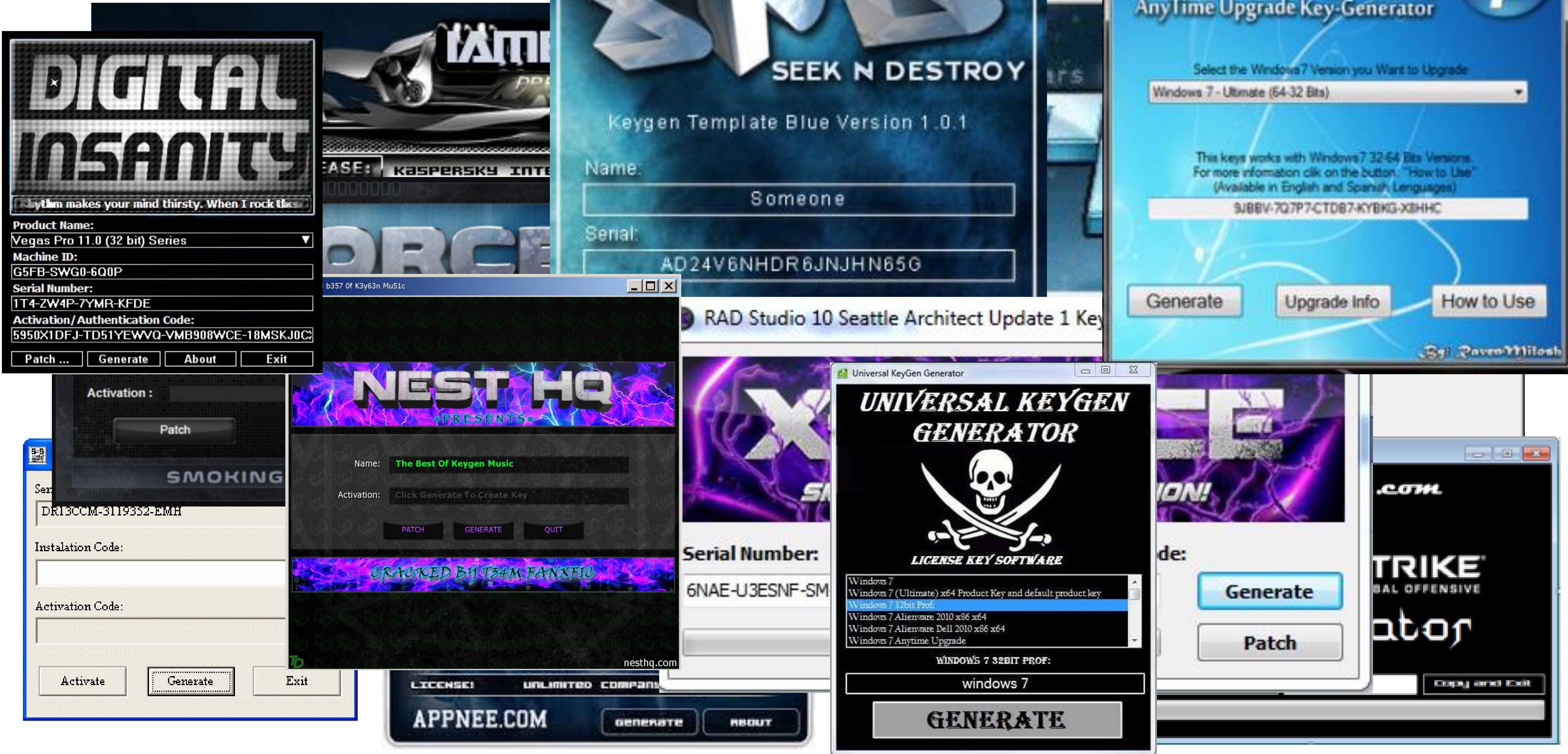
Reverse Engineering

Software License checkers

- Offline DRM
- Have a secret algorithm that takes in input, performs some calculation, and validates the result.
- Ideally, the company selling the software can generate multiple valid keys.
- Ideally, pirates cannot generate valid keys.

Implicitly trusts the binary code
to keep the secret license key verification algorithm safe!

The



The Rise of Keygens



Reverse Engineering - Simple Example

- Crackmes are small programs designed to test a hacker's reverse engineering skillz.
- Inspired by real-world license key verification systems.

Many many crackmes: <https://crackmes.one>

Evolution of DRM

- **Alternative:**
 - Cracks that patch the executable to remove the check altogether (but what to crack?).
- **Modern DRM has taken on more and more clever approaches:**
 - **Code Obfuscation:** make code difficult to read
 - **Anti-debugging:** techniques to fight dynamic analysis
 - **VMing:** wrapping the DRM code in a heavily obfuscated, protected emulator for a custom architecture, with the DRM logic inside that architecture.
 - **Trusted Execution Environments:** moving the DRM outside of the CPU into protected hardware.

Code Obfuscation

- **Renaming**

```
def calculate_sum(a, b):  
    result = a + b  
    return result
```

```
def x1(x2, x3):  
    x4 = x2 + x3  
    return x4
```


Code Obfuscation

- **Control flow Obfuscation**

```
def is_even(n):  
    return n % 2 == 0
```

```
def is_even(n):  
    if n > 100 or n < 100:  
        if n % 2 == 0:  
            return True  
        else:  
            return False
```

Code Obfuscation

- **String Encoding**

```
message = "Hello, World!"
```

```
import base64
encoded_message = "SGVsbG8sIFdvcmxkIQ=="
message = base64.b64decode(encoded_message).decode('utf-8')
```

Code Obfuscation

- **Code inlining**

```
def square(x):  
    return x * x  
  
print(square(5))
```

```
x = 5  
print(x * x)
```


Code Obfuscation

- **Dead Code Insertion**

```
def multiply(a, b):  
    return a * b
```

```
def multiply(a, b):  
    unused = a + b # This code does nothing  
    return a * b
```


Code Obfuscation

- **Opaque Predicates**

```
def add(a, b):  
    return a + b
```

```
def add(a, b):  
    if a * 2 == a + a: # This condition is always true  
        return a + b  
    else:  
        return 0
```

Code Obfuscation

- **Flattening**

```
def calc(x):  
    if x > 0:  
        return x * 2  
    else:  
        return x - 2
```

```
def calc(x):  
    table = [lambda: x * 2, lambda: x - 2]  
    index = 0 if x > 0 else 1  
    return table[index]()
```

Code Obfuscation

- **Unrolling Loops**

```
for i in range(3):  
    print(i)
```

```
i = 0  
print(i)  
i = 1  
print(i)  
i = 2  
print(i)
```


Reverse Engineering

Game Modding

- Adding custom features to games
- Cheat engines: Aim mods, ...



Fallout 4



BeamNG.drive



Sims 4



Street Legal Racing Redline



GTA 5



GTA 4



GTA San Andreas



GTA Vice City



TES 5: Skyrim



Fallout New Vegas



Counter-Strike Source



Counter Strike 1.6



Euro Truck Simulator 2



Farming Simulator 2013



GTA 3



World Of Tanks